# 5 Data Tables Homework

Sai Ram Vodnala

06/25/2024

There are four exercises below. Three exercises are worth 10 points each, and are labelled (Either R, Python or Julia). For these exercises, you can implement a solution in the language of your choice. I've included code chunk templates for all three languages, but you only need to fill in the chunks for one language.

One exercise is labelled (R, Python and Julia). This exercise is worth 20 points, and you must provide a solution using all three languages. This exercise also includes code chunk templates for all three languages.

Some of the exercises have questions relating to the results. Write your answers in the narrative part of the text, formatted as *italicized* text, **bold** text or

> quoted text.

Submit both `Rmd` and typeset results using file name formats as in previous exercises.

Do not use libraries in R that are not part of the base R installation; this includes popular libraries for data tables like `data.table` or tidyverse libraries like `dplyr` and `tidyr`. You may use any Python or Julia libraries introduced in lecture, except for graphs. You are free to use any Python or Julia graphics library, as long as I can install it on my machine if necessary.

## Exercise 1. (R, Python or Julia)

The OzDASL web site includes a data set titled **Pain Thresholds of Blonds and Brunettes** (http://www.statsci.org/data/oz/blonds.html). The data are as follows:

| HairColour | Pain |
|---|---|
| LightBlond | 62 |
| LightBlond | 60 |
| LightBlond | 71 |
| LightBlond | 55 |
| LightBlond | 48 |
| DarkBlond | 63 |
| DarkBlond | 57 |
| DarkBlond | 52 |
| DarkBlond | 41 |
| DarkBlond | 43 |
| LightBrunette | 42 |
| LightBrunette | 50 |
| LightBrunette | 41 |
| LightBrunette | 37 |
| DarkBrunette | 32 |

| HairColour | Pain |
|---|---|
| DarkBrunette | 39 |
| DarkBrunette | 51 |
| DarkBrunette | 30 |
| DarkBrunette | 35 |

## Part a.

Copy the data from the table above to create a `data.frame` in R, a `pandas dataframe` in Python, or a `DataFrame` in Julia. Write the code the appropriate chunk below. Name the data table `PainThreshold`. Note that the values in column one will need to be coded as text (with single or double quotes as necessary). Don't download the linked text file; this is an exercise in programmatically creating a data table. Take care to format the data table code so that it fits in the typeset document.

After creating the table, modify `HairColour` so that it is an ordinal variable, with the order `LightBlond`, `DarkBlond`, `LightBrunette` and `DarkBrunette`.

```
PainThreshold <- data.frame(
HairColour = c("LightBlond", "LightBlond", "LightBlond", "LightBlond", "LightBlond",
               "DarkBlond", "DarkBlond", "DarkBlond", "DarkBlond", "DarkBlond",
               "LightBrunette", "LightBrunette", "LightBrunette","LightBrunette","DarkBrunette","Dark
  Pain = c(62, 60, 71, 55, 48, 63, 57, 52, 41, 43, 42, 50, 41, 37, 32, 39, 51, 30, 35)
)

PainThreshold$HairColour <- factor(PainThreshold$HairColour, levels = c("LightBlond", "DarkBlond", "Lig
```

### R

### Python

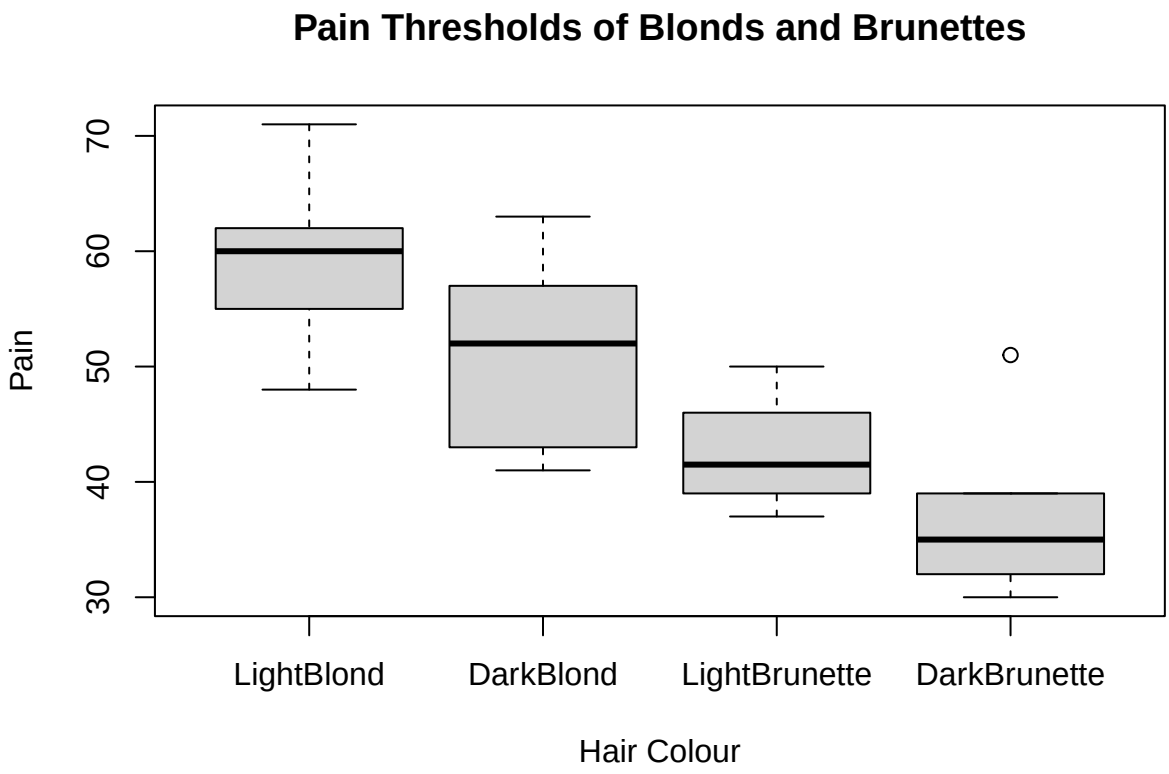
### Julia

## Part b.

The linked source for these data includes a box-whisker plot, with `HairColour` as the independent variable. Reproduce the box-whisker plot below. Did you enter the data correctly? Compare with the source plot.

*Yes, I entered the data correctly but compared to the source plot the below plot shows it as an oulier for DarkBrunette.*

```
boxplot(PainThreshold$Pain ~ PainThreshold$HairColour, data = PainThreshold,
xlab = "Hair Colour", ylab = "Pain", main = "Pain Thresholds of Blonds and Brunettes")
```




**R**

**Python**

**Julia**

## Exercise 2. (R, Python or Julia)

You will be asked to use your `ConfidenceInterval` function for this exercise. Include the function definition here.

```r
# function definitions
ConfidenceInterval <- function(data, confidence_level = 0.95) {
  n <- length(data)
  mean_val <- mean(data)
  std_err <- sd(data) / sqrt(n)
  z <- qnorm((1 + confidence_level) / 2)

  lower_bound <- mean_val - z * std_err
  upper_bound <- mean_val + z * std_err

  return(c(lower_bound, upper_bound))
}
```

**R**

**Python**

**Julia**

## Part a

Go to http://www.itl.nist.gov/div898/strd/anova/SiRstv.html and use the data listed under `Data File in Table Format` (https://www.itl.nist.gov/div898/strd/anova/SiRstvt.dat)

## Part b

Edit this into a file (tab delimited, `.csv`, etc,) that can be read into R, Python or Julia, or find an appropriate function that can read the file as-is. You will need to upload the edited file to D2L along with your Rmd files. Provide a brief comment on changes you make, or assumptions about the file needed for you file to be read into R, Python or Julia. Read the data into a data table.

*I copied the data values from a website, then converted them into a DataFrame using pandas, and saved it as a CSV file in Jupyter Notebook. Note: I also attached the .ipynb and .csv files in D2L.*

```
SiRstvt <- read.csv("C:/Users/sairam/Desktop/Stat600/0.files/HOMEWORKS/HW5/SiRstv.csv", header = TRUE)
print(SiRstvt)
```

**R**

```
##         X1       X2       X3       X4       X5
## 1 196.3052 196.3042 196.1303 196.2795 196.2119
## 2 196.1240 196.3825 196.2005 196.1748 196.1051
## 3 196.1890 196.1669 196.2889 196.1494 196.1850
## 4 196.2569 196.3257 196.0343 196.1485 196.0052
## 5 196.3403 196.0422 196.1811 195.9885 196.2090
```

**Python**

**Julia**

## Part c.

There are 5 columns in these data. Calculate mean, standard deviation and sample size for each column in this data, using column summary functions. Print the results below.

```
mean1 <- sapply(SiRstvt, mean)
print(mean1)
```

**R**

```
##       X1       X2       X3       X4       X5
## 196.2431 196.2443 196.1670 196.1481 196.1432
```

```
sd1 <- sapply(SiRstvt, sd)
print(sd1)
```

```
##         X1         X2         X3         X4         X5
## 0.08747329 0.13797498 0.09372413 0.10422674 0.08844797
```

```
n1 <- sapply(SiRstvt, length)
print(n1)
```

```
## X1 X2 X3 X4 X5
##  5  5  5  5  5
```

Reuse your `ConfidenceInterval` function to compute confidence intervals for the means in this data set. Note, you can do this with one function call if you use vectors, list comprehensions or broadcasting.

```
SE<- function(sd,n){
  sd/sqrt(n)
  }


ConfidenceBound <- function(sd, n, alpha=0.05) {
qnorm(1-alpha/2)*SE(sd,n)
  }

ConfidenceInterval_distribution <- function(mean,sd,n) {
lower<- (mean - ConfidenceBound(sd,n))
upper<- (mean + ConfidenceBound(sd,n))
return(list(Lower_Ndistribution=lower, Upper_Ndistribution=upper))
}
confidence_intervals <- ConfidenceInterval_distribution(mean1, sd1, n1)

print(confidence_intervals)
```

**R**

```
## $Lower_Ndistribution
##       X1       X2       X3       X4       X5
## 196.1664 196.1234 196.0849 196.0568 196.0657
##
## $Upper_Ndistribution
##       X1       X2       X3       X4       X5
## 196.3198 196.3652 196.2492 196.2395 196.2208
```

## Exercise 3 (R, Python or Julia)

We will use data from https://acsess.onlinelibrary.wiley.com/doi/abs/10.2134/jeq2007.0099, Table 1. The original paper is also available on D2L.

Download the file `Khan.csv` from D2L and read the file into a data frame. Print a summary of the table.

```
data <- read.csv("C:/Users/sairam/Desktop/Stat600/0.files/HOMEWORKS/HW5/Khan.csv")
summary(data)
```

**R**

```
##    Rotation          Fertilizer           Depth              Mean55
##  Length:27          Length:27          Length:27          Min.   :1.020
##  Class :character   Class :character   Class :character   1st Qu.:1.434
##  Mode  :character   Mode  :character   Mode  :character   Median :1.487
##                                                           Mean   :1.538
##                                                           3rd Qu.:1.661
##                                                           Max.   :2.109
##      Mean05           SD05               Diff
##  Min.   :0.751   Min.   :0.004000   Min.   :-0.5020
##  1st Qu.:1.141   1st Qu.:0.006500   1st Qu.:-0.2970
##  Median :1.312   Median :0.008000   Median :-0.2090
##  Mean   :1.325   Mean   :0.008519   Mean   :-0.2126
##  3rd Qu.:1.474   3rd Qu.:0.010000   3rd Qu.:-0.1105
##  Max.   :1.887   Max.   :0.014000   Max.   : 0.0130
```

To show that the data was read correctly, create three plots. Plot

1. Rotation vs Fertilizer
2. Mean55 vs Fertilizer
3. Mean55 vs Mean05

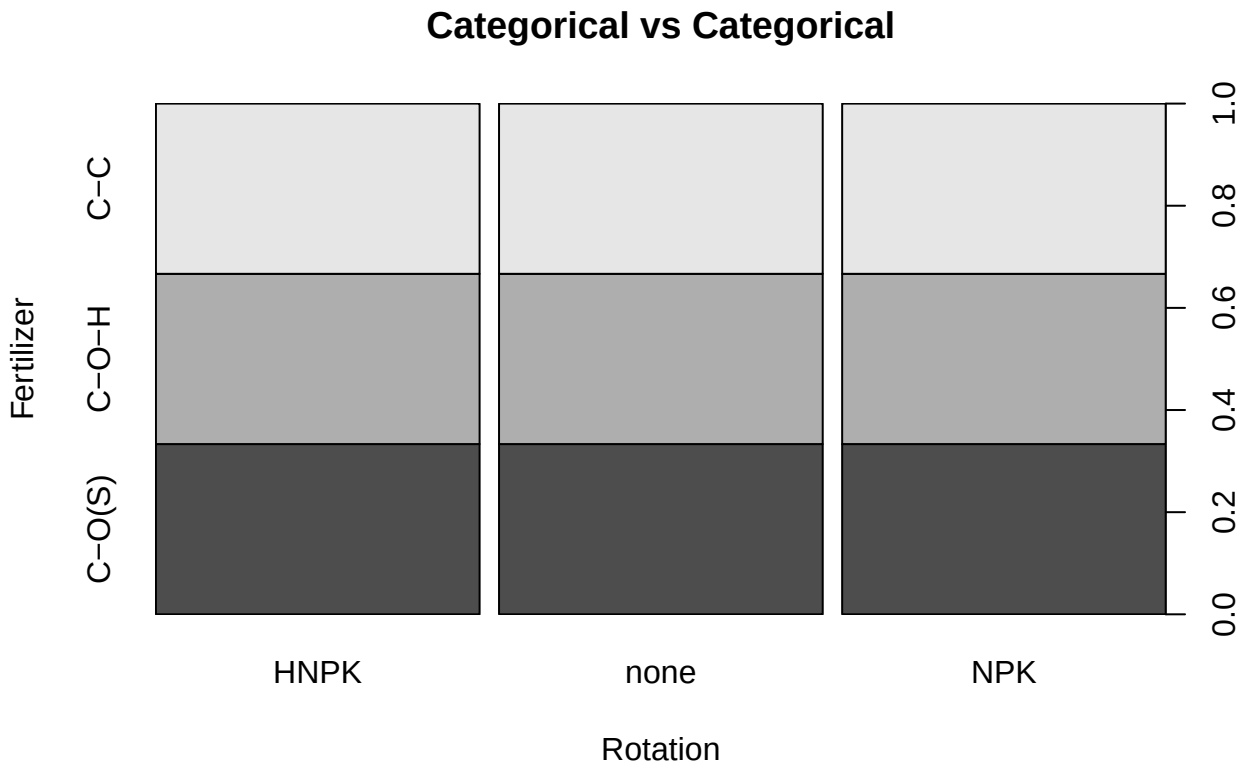
`Mean05` and `Mean55` are the amount of soil organic carbon measured in crop land experimental units in 2005 and 1955 respectively. `Rotation` is the crop rotation plan (i.e. corn followed by soybeans followed by corn) for the respective plots, and `Fertilizer` is the type of fertilizer applied to the plots over the period from 1955-2005.

These three plots should reproduce the three types of plots shown in the `Week 5 Data Tables Plots` video, **Categorical vs Categorical**, **Continuous vs Continuous** and **Continuous vs Categorical**. Add these as titles to your plots, as appropriate. If you choose Julia for this exercise, you will not be required to plot **Categorical vs Categorical**.

Do you notice anything unusual about the data?

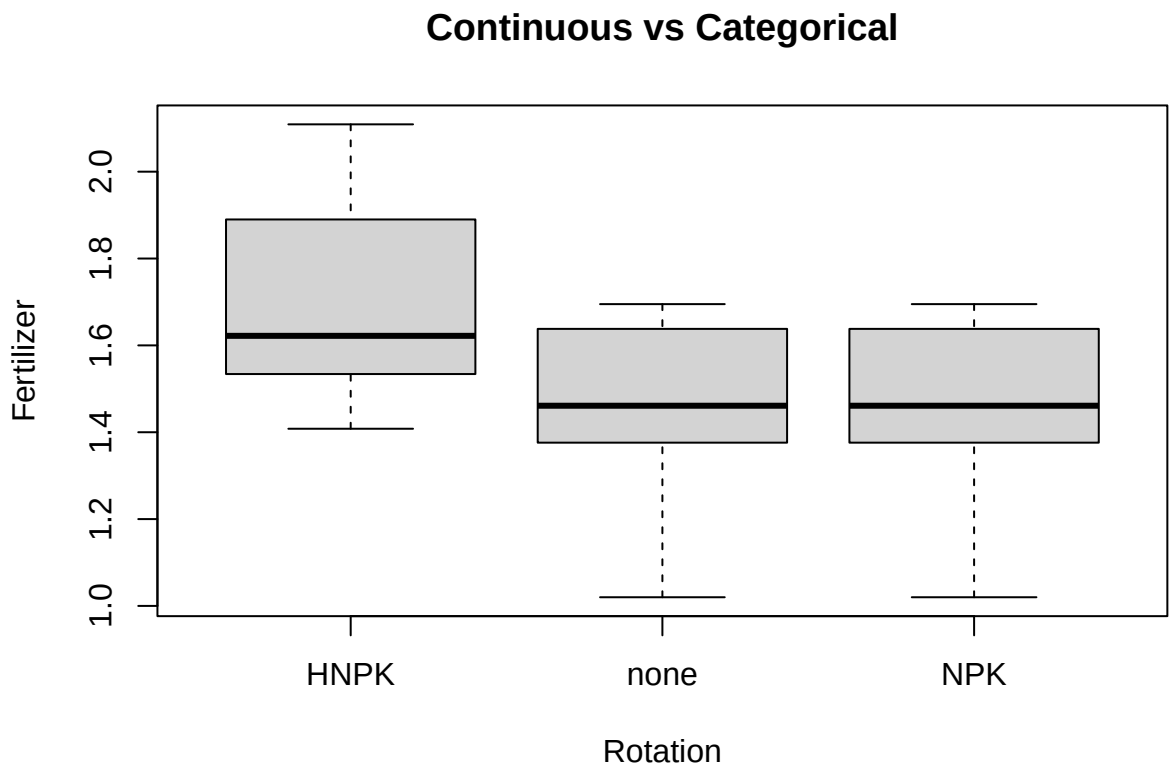
*In the box plot (Continuous vs. Categorical), the distributions of Mean55 for different fertilizer types do not show a large variation. This could indicate that the type of fertilizer does not have a strong influence on the Mean55 value and also In scatter plot (Continuous vs. Continuous) shows a relatively weak correlation between Mean55 and Mean05.*

```r
data$Rotation <- as.factor(data$Rotation)
data$Fertilizer <- as.factor(data$Fertilizer)
plot(data$Rotation ~ data$Fertilizer,ylab = 'Fertilizer',xlab ='Rotation', main = "Categorical vs Categ
```

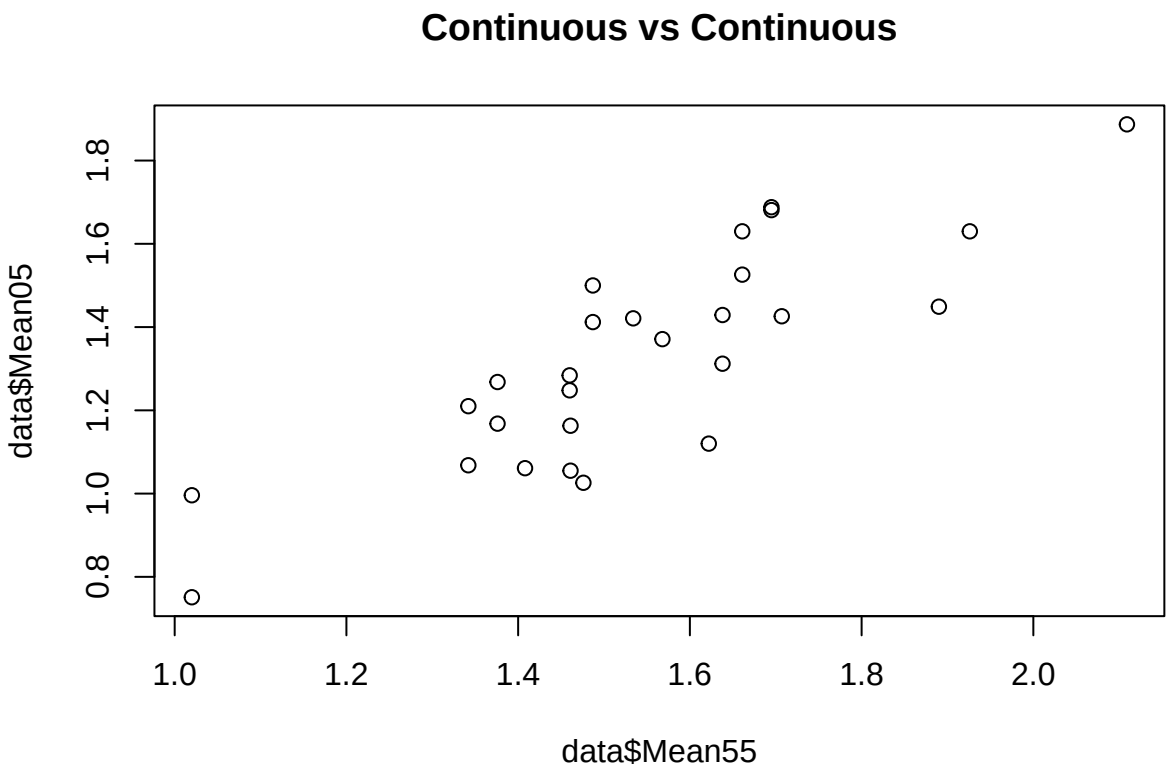



**R**

```r
plot(data$Mean55 ~ data$Fertilizer,ylab = 'Fertilizer',xlab ='Rotation', main = "Continuous vs Categori
```




```
plot(x = data$Mean55,y = data$Mean05,type = 'p',main = "Continuous vs Continuous" )
```




## Exercise 4. (R, Julia and Python)

### Part a.

Example data for the chemical composition of glass fragments are available https://archive.ics.uci.edu/ml/datasets/Glass+Identification. I've uploaded the data to D2L in the file `glass.data`. This is in CSV format with no header. Read the data into R, Python or Julia.

Add the following column headers to the data table (see the source link for a description of the data columns).

1. `Id`
2. `RI`
3. `Na`
4. `Mg`
5. `Al`
6. `Si`
7. `K`
8. `Ca`
9. `Ba`
10. `Fe`
11. `Type`

You should not print the data tables in the typeset document. Instead, steps in Part b will tell us if the columns have been properly renamed. However, you should print the number of rows in the data. There should be 214 rows and 11 columns.

```r
column_names <- c('Id','RI','Na','Mg','Al','Si','K','Ca','Ba','Fe','Type')
glass <- read.csv("C:/Users/sairam/Desktop/Stat600/0.files/HOMEWORKS/HW5/glass.data", header = FALSE, co

cat('Number of rows:', nrow(glass), '\n')
```

**R**

```
## Number of rows: 214
```

```r
cat('Number of columns:', ncol(glass), '\n')
```

```
## Number of columns: 11
```

```python
import pandas as pd
data = pd.read_csv("C:/Users/sairam/Desktop/Stat600/0.files/HOMEWORKS/HW5/glass.data", header=None, name
print("Number of rows:", data.shape[0])
```

**Python**

```
## Number of rows: 214
```

```python
print("Number of columns:", data.shape[1])
```

```
## Number of columns: 11
```

```julia
import Pkg
Pkg.add("CSV")
using CSV, DataFrames

data = CSV.read("C:/Users/sairam/Desktop/Stat600/0.files/HOMEWORKS/HW5/glass.data", header=false, DataFr
colnames = ["Id", "RI", "Na", "Mg", "Al", "Si", "K", "Ca", "Ba", "Fe", "Type"];
rename!(data, Symbol.(colnames));
n_rows = size(data, 1);
println(n_rows)
```

**Julia**

```
## 214
```

```julia
n_columns = size(data, 2);
println(n_columns)
```

```
## 11
```

## Part b

Create two new data columns for $\log(Ca/K)$ and $\log(Ca/Si)$. The first is the log of (column `Ca` divided by column `K`), the second is the log of (column `Ca` divided by column `Si`). You don't need to use the column names `log(Ca/K)` or `log(Ca/Si)`, you can use names without special characters (i.e. '(' or '/')). You should not print the data tables in the typeset document. We'll check the calculations in Part c.

```
glasslog_Ca_K <- log(glass$Ca / glass$K)
glasslog_Ca_Si <- log(glass$Ca / glass$Si)
```

**R**

```
# Create the new columns
import numpy as np
data["log_Ca_K"] = np.log(data["Ca"] / data["K"])
data["log_Ca_Si"] = np.log(data["Ca"] / data["Si"])
```

**Python**

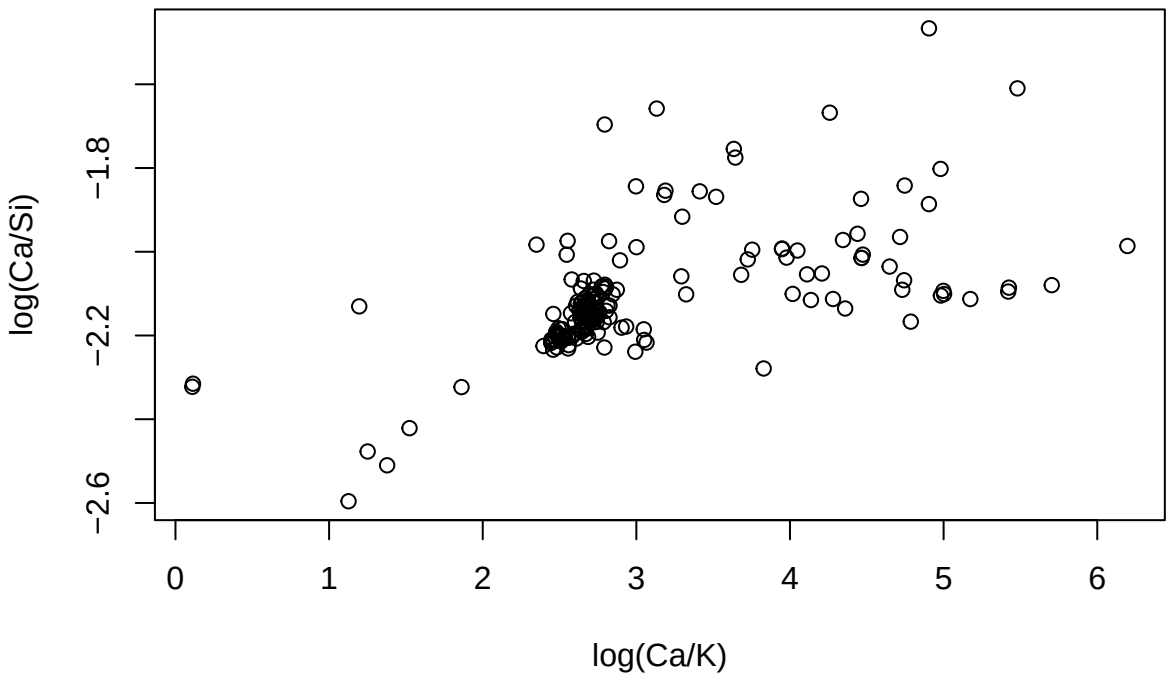
```
using DataFrames
data.log_k = log.(data.Ca ./ data.K);
data.log_si = log.(data.Ca ./ data.Si);
```

**Julia**

## Part c

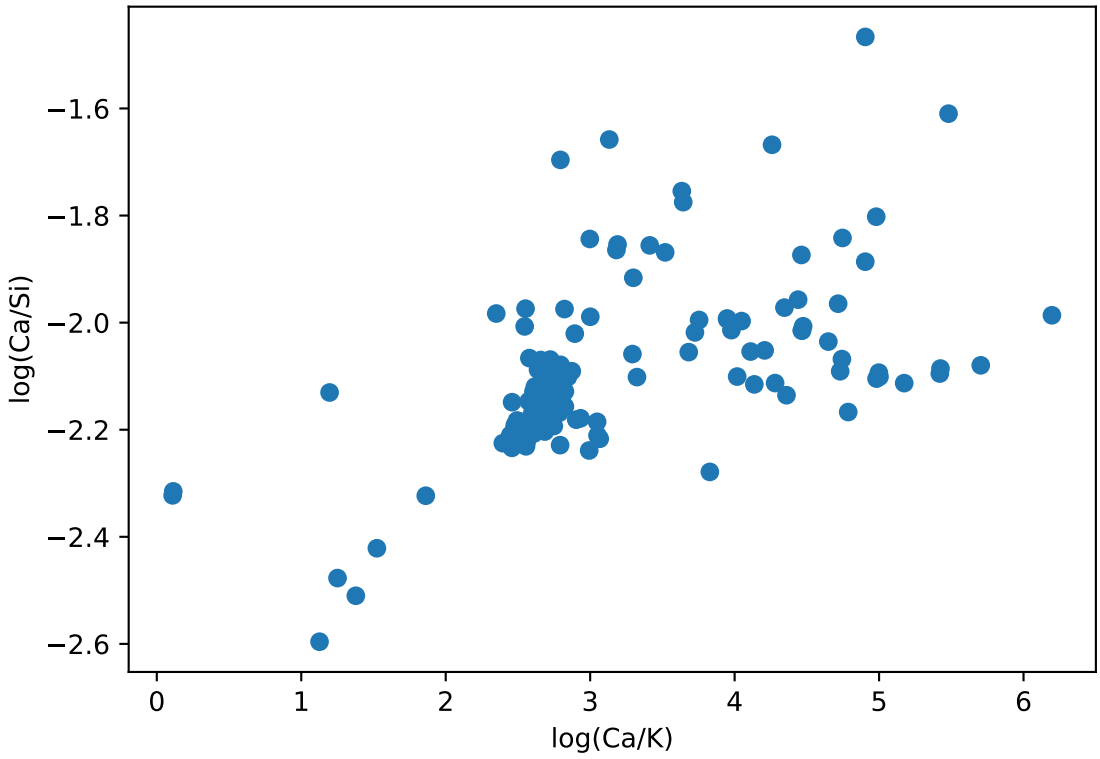
To show your work, plot `log(Ca/Si)` versus `log(Ca/K)` (independent variable). The x- and y-axes of your plot should be labelled `log(Ca/K)` and `log(Ca/Si)`.

```
plot(glasslog_Ca_K, glasslog_Ca_Si, xlab = "log(Ca/K)", ylab = "log(Ca/Si)")
```



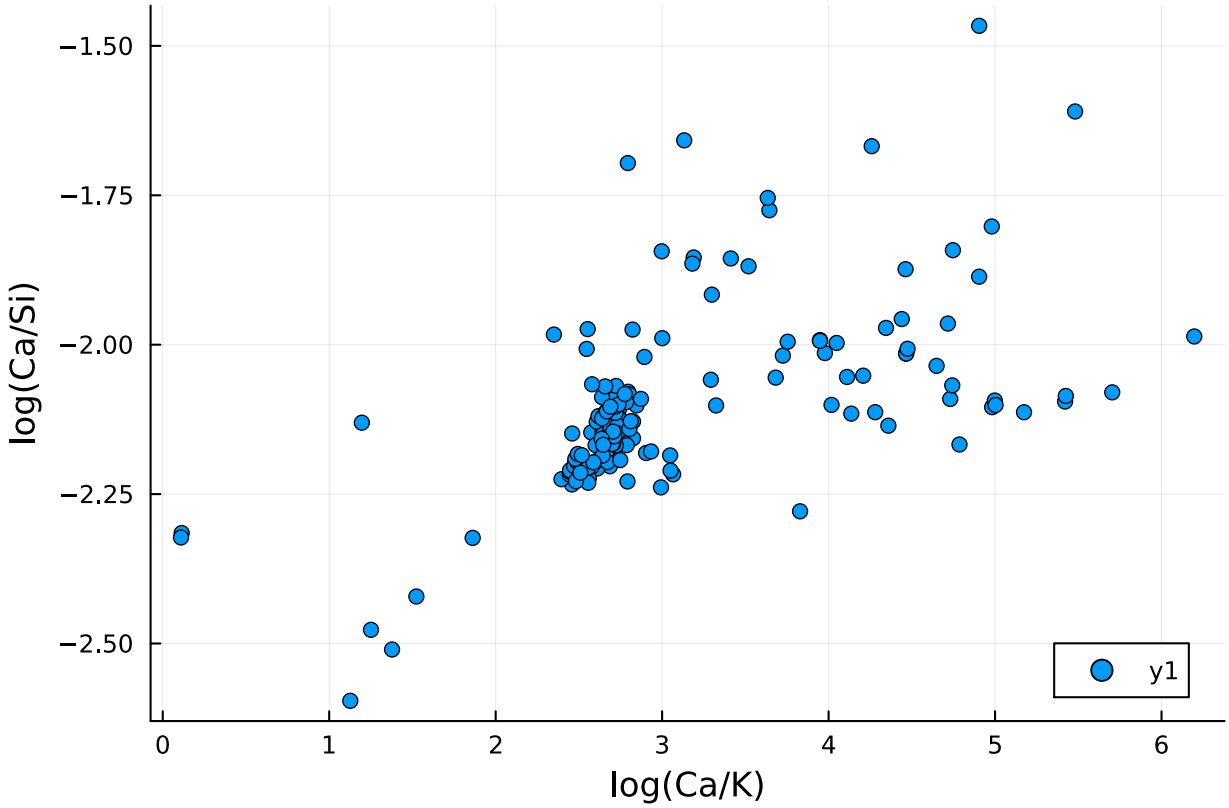

**R**

```
import matplotlib.pyplot as plt
plt.scatter(data["log_Ca_K"], data["log_Ca_Si"])
plt.xlabel("log(Ca/K)")
plt.ylabel("log(Ca/Si)")
plt.show()
```




**Python**

```
using Plots
scatter(data.log_k, data.log_si, xlabel = "log(Ca/K)", ylabel = "log(Ca/Si)")
```




**Julia**

As a side note, Dr. Saunders has been using a plot similar to this on the homepage for STAT 601 and STAT 602.